2025 IEEE International Conference on Cyber Security and Resilience (CSR) | 979-8-3315-3591-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CSR64739.2025.11130040

Simulation-Based Analysis of OPC UA Set-Up Vulnerability and Its Security Risks in Cyber-Physical Manufacturing Systems

Sani M. Abdullahi[†], Manuel Götz[‡], Sanja Lazarova-Molnar[‡†]
[†]*Mærsk McKinney Møller Institute, University of Southern Denmark, Campusvej 55, Odense, 5230, Denmark*
[‡]*Institute AIFB, Karlsruhe Institute of Technology, Kaiserstr. 89, Karlsruhe, 76133, Germany*
{saa, slmo}@mmmi.sdu.dk; manuel.goetz@kit.edu; lazarova-molnar@kit.edu

***Abstract*—In the era of Industry 4.0 (I4.0), Cyber-Physical Systems (CPS) are pivotal in enhancing industrial processes by using the Open Platform Communication Unified Architecture (OPC-UA) as the standard communication protocol. OPC-UA is one of the few industrial protocols that incorporates security measures for preventing adversaries from compromising critical infrastructures. However, despite the protocol's security features and widespread adoption, it continues to reveal cybersecurity challenges and setup vulnerabilities in several products and vendor implementation libraries. Many vendor implementations of OPC-UA are unaware of the security risks associated with non-compliance to the OPC-UA specification or the integration of OPC-UA with other insecure protocols. Therefore, this work demonstrates the weaknesses of OPC-UA in such vendor implementation contexts by analyzing the communication architecture of the adopted OPC-UA library. Specifically, we start by simulating a communication architecture set-up that mimics most vendor OPC-UA implementations, which includes Digital Twin, Cobots, Manufacturing Execution Systems (MES) and Enterprise Resource Planning (ERP) systems with OPC-UA as the main protocol between MES and Cobots. A suspicious Media Access Control (MAC) address is sniffed in the network, and a malicious packet is then injected by exploiting the final TCP layer of the OPC-UA rogue client closing session, which rendered the entire network vulnerable. To verify the security risk in our communication architecture, we investigate the impact of the vulnerability in the simulation setup by instigating a Denial of Service (DoS) attack through the gradual increase in the number of malicious packets at different loops while noting how the systems react. This reveal flaws in the system operation, ranging from unreliable data exchange to total system collapse. Finally, different mitigation strategies are proposed to avert the attack.**

Keywords— cyber-physical systems, OPC-UA, vulnerability, digital twins, cobots, DoS

I. Introduction

The modern Industry 4.0 environment has transformed industrial processes by integrating information technology (IT) and operational technology (OT) via CPS, resulting in remarkable efficiency and automation [1]. This transition increased interconnection between different components, thus exposing systems to many cybersecurity challenges, especially in vulnerabilities inherent to communication protocols like Modbus TCP [2]. The Open Platform Communication-Unified Architecture (OPC-UA), now considered the standard protocol for enabling unified communications in industrial settings, was first released by the OPC Foundation with specifications including security features like authentication, authorization, integrity, and confidentiality [3]. OPC-UA's invention came as a promising advancement in security of communication protocols given the plethora of security vulnerabilities in other preceding protocols [4, 5]. Moreover, as more manufacturers develop new products that use OPC-UA or upgrade existing devices to support the protocol, it emerges as one of the handful of technologies in the industry that are universally recognized and embraced [6].

OPC-UA is recognized for both enhancing security and overcoming crucial interoperability challenges among different IT and OT technologies that were previously unable to efficiently communicate data. It is also considered as the established protocol that enables operators and engineers to connect with and control physical operations on the OT network via a single server [7]. As such, OPC-UA's platform independence enables devices from many manufacturers to connect easily, which makes the OPC-UA a critical protocol. However, substantial challenges such as information leakage and remote code execution [3, 8] still impede the establishment of secure OPC-UA implementations in reality. This is exacerbated by several factors, which include invalid security configurations on the OPC-UA servers, insecure OPC-UA products and libraries, protocols not conforming to the OPC-UA foundation standards, misconfigurations in the OPC-UA implementation, a lack of compliance certificates for security features (especially in open-source implementations), etc.

Recently, researchers have been analyzing and reporting more vulnerabilities in various stacks of OPC-UA products and libraries. The Team82 authors from Clatory developed an advanced OPC-UA exploit framework [8] that is used to execute numerous attacks against OPC-UA implementations. Sharon et al. identified almost 50 vulnerabilities in different products that rely on the OPC-UA protocol, including clients, servers, and protocol gateways. The exploit framework comprises four categories of payloads: sanity, attacks, corpus,

and server, each capable of executing exploits. The different variants of uncovered vulnerabilities include DoS, information leakage, and remote code execution.

In [3], Erba et al. conducted a security analysis on implementations of different OPC-UA products and libraries. The investigation is conducted on 48 open-source OPC-UA products and libraries, of which 38 exhibit one or more security issues, such as lack of support for security features, misconfigurations, erroneous instructions, lack of security compliance, etc. The authors further verify their analysis by demonstrating attacks such as rogue client, rogue server, eavesdropping, and data tampering.

Neu et al. [9] simulated a DoS attack in OPC-UA networks and proposed an approach for detecting such attacks. The simulation mainly focused on two different DoS attack-based impacts: DoS attacks affecting only the network and DoS attacks affecting both the network and the CPU of the OPC-UA server. To detect these attacks, the authors extracted features from the analyzed packets inside the simulated datasets of the OPC-UA communication network. The features were extracted by classifying packets and establishing a typical OPC-UA packet flow pattern on the network. The attack is capable of becoming distributed when the SignAndEncrypt mode in encrypted malicious packets is used on a large number of devices.

In [10], the authors investigate the security and scalability of open-source OPC-UA implementations given their large number of deployments. Specifically, the authors examine the security frameworks of four most prevalent open-source OPC-UA implementations, including python-opcua, open62541, node-opcua, and UA-.NET Standard, based on code review. Their investigation reveals some security flaws in their deployment, including the absence of compliance certificates, a missing timeout in python-opcua, and less stringent packet checking in node-opcua. Their security models fail to fully comply with the foundation's specifications. In terms of scalability, their analysis reveals that the choice of programming language plays a major role in the capacity of the OPC-UA server to handle the anticipated demand and the number of resources it takes. Overall, open62541 and UA-.NET Standard are considered better in terms of both security and scalability.

In general, the OPC UA protocol is meant to be safe, scalable, secure, and platform-independent, which makes it an excellent choice for use in control systems and industrial automation. Because of its adaptable architecture, it can be employed in a diverse array of applications, ranging from embedded systems on a small scale to business settings on a much larger scale [11]. However, uncovering such vulnerabilities reveal that a single attack on any variant of OPC-UA artifact can lead to irreparable damage to a wide range of OPC-UA products and consequently all industries adopting those products.

In this paper, we further analyzed one of the widely adopted open-source OPC-UA implementations, the python-opcua [12], within a simulated network architecture. In the analysis, we mimicked the communication network between four different entities: Digital Twin (DT), ERP, MES, and Cobots, with OPC-UA as the main protocol between MES and Cobots. We then sniffed a suspicious Media Access Control (MAC) address because it carries a packet that is not intended for the server. A malicious packet is then injected into the communication loop in place of a genuine packet sniffed from the MAC address. This distorts the entire communication by sending unreliable data between entities. Additionally, the analysis reveals that sending a high number of malicious packets at multiple loops disrupts the entire communication and crashes the system operations. To avert this vulnerability, several mitigation strategies are recommended.

The contributions of this work are summarized as follows:

1) We simulated a communication architecture that mimics regular vendor implementations with four main crucial elements of CPS in the manufacturing industry, including DT, Cobots, ERP, and MES.
2) We thoroughly analyzed the OPC-UA communication between MES and Cobots—the adopted library for such communications, as well as the entire communication architecture—and how all these led to a vulnerability.
3) We uncovered a DoS attack by injecting malicious packets in the communication loop of the closing session and further validated the feasibility of the uncovered attack by analyzing its impact in three different scenarios.
4) Finally, we proposed different mitigation strategies to avert this vulnerability.

To the best of our knowledge, our communication setup is the aiding mechanism for the DoS attack, which mimics flaws in existing vendor implementations. As such, the attack feasibility depends on our target system (simulated communication architecture) and does not pertain to any particular software or hardware flaw. Therefore, neither the OPC foundation nor any Python-OPC-UA vendors have received any disclosure.

The remaining part of this work is organized as follows: In Section 2, we present the background on communication in manufacturing industries including the OPC-UA communication protocol. In Section 3, we discussed our simulation set-up. In Section 4, we delve into the main attack design, beginning with an analysis of the OPC-UA communication protocol and the attack strategy. Section 5 provided a comprehensive analysis of the impact of the attack on data unreliability and system disruption and proposed various mitigation strategies to prevent the attack. Finally, the conclusion and future work are drawn in Section 6.

II. Background

In this Section, we introduce the automation pyramid in Section 2.1 and the communication protocol OPC-UA in Section 2.2.

A. Communication Manufacturing

In manufacturing, efficient communication between the different layers of the production hierarchy is essential for maintaining smooth operations. The automation pyramid, shown in Figure 1, provides a structured framework to manage these interactions, ranging from shop-floor machines to high-level business systems [13]. Facilitating real-time data exchange helps optimize decision-making processes and overall production quality and efficiency.

At Level 0, actual production activities occur, such as machining and assembly. This level involves physical actions, aided by sensors and actuators, which are part of Level 1. For

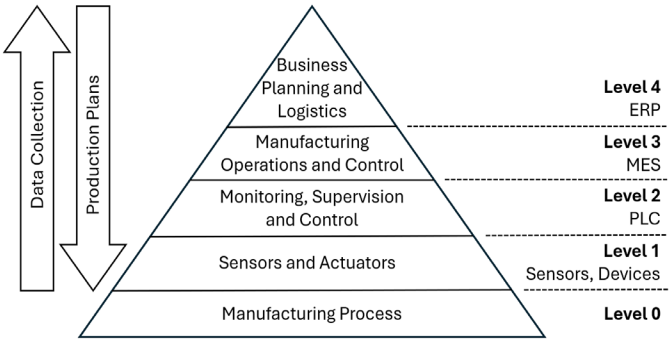

Fig. 1. The automation pyramid

example, in automated assembly, a robotic arm might act as an actuator, executing precise movements based on sensor input. Level 2 oversees and controls the operations of lower levels. Programmable Logic Controllers (PLCs) are used here to manage the interaction between sensors and actuators. At Level 3, Manufacturing Operations and Control (MO&C) focuses on managing workflows, part lists, and product recipes. It handles dynamic job scheduling, process optimization, and data management, often facilitated by Manufacturing Execution Systems (MES). Level 4 is responsible for business planning and logistics. This level covers strategic activities such as material planning, inventory management, and production scheduling, typically managed by Enterprise Resource Planning (ERP) systems [14].

Effective communication between these levels ensures that production goals align with business objectives and allows real-time adjustments based on data from shop-floor machines and cobots. In Section 3, we present our implementation of the automation pyramid for our Use Case, outlining the technologies used and the rationale behind their selection.

With this, the importance of communication in the entire production process cannot be overemphasized as the transmission of all vital information is performed via these processes. Therefore, it is imperative to ensure all manufacturing industries are built on communication protocols that are designed to endure the challenges of the OT environment, which is characterized by elevated electrical noise, the need for scalability, reliability, low latency, optimal performance, and security [15].

B. The Communication Protocol OPC-UA

The OPC-UA protocol is built based on different layers of communication and information exchange. This includes the application layer, session layer, and transport layer [11]. The **application layer** is responsible for defining the data types and services that are transmitted between clients and servers in the OPC-UA. It also encompasses the object mode of the OPC-UA, which delineates the framework and functional behavior of the data available for exchange. The **session layer** is responsible for overseeing the interactions between OPC-UA clients and servers. It manages the initialization, coordination, and completion of communication sessions. Lastly, the **transport layer** streamlines data transfer between clients and servers within the OPC-UA. It contains a variety of data transport protocols, such as TCP/IP and HTTPS. While it is possible to use the OPC-UA protocol via a variety of transport layers, including HTTP, TCP/IP, and MQTT, its most popular use is TCP/IP [16]. However, it is not possible to choose a single predetermined port; rather, every application is responsible for selecting its own distinct port.

At a higher level, the OPC-UA message structure begins with an ASCII code of three-byte, which establishes the message format in four distinct categories. These include the HEL, OPN, MSG, and CLO. The HEL message (Hello message) is the first conveyed message. A number of fundamental parameters are included in the HEL message in order to initiate the connection. These include the URL that the client wants to establish a connection to and the maximum packet that the client anticipates receiving. Next, in the event that everything goes well, OPN (OpenSecureChannel) establishes a secure channel via the same connection, and the server replies with a SecureCahnnelID. Hence, the client and server communicate securely over the established channels. An often-used transmitted MSG (Generic message container) is the Browse, which the client employs to get the node hierarchy originating from the server. After communicating the MSG, the session concludes with the CLO (CloseSecureChannel) message, which signifies its termination. Fig. 3 depicts the sample OPC-UA communication flow in a single session. For more information on the data modeling and encoding functionality of the OPC-UA, readers can refer to [17].

Given the significance of the transport layer in our DoS attack, it is crucial to clarify its role in OPC-UA, as it plays a pivotal role in the overall communication protocol. It is the responsibility of the transport layer to ensure that data is sent between OPC-UA clients and servers via a variety of transmission protocols. Also, all the OPC-UA connections from initialization to termination between clients and servers are managed by the transport layer, which runs on top of the network layer. Given that the proposed attack aims to halt, significantly disrupt, or interfere with the data transmission within the assembly line, the easiest way to accomplish this objective is to directly engage with the essential layers responsible for data transportation and reliable end-to-end communication within the entire protocol, which is the TCP/IP. Taking this into account, we now formulate the following attack strategy.

III. Simulation Model Design and Component

In this section, we provide an overview of the design and components of our simulation model, which is used to examine the interactions between key elements of a modern production system. The model leverages the automation pyramid framework to streamline communication between different systems, including ERP, MES, DT, and Cobots. We focus on simulating the communication flows between these systems to identify potential cybersecurity vulnerabilities, particularly those concerning the OPC-UA communication protocol. The subsequent subsections describe the architecture of the model, the communication framework, and the technical details of the setup used in the simulation.

A. Communication Framework and Data Flow

In the following we describe how we implemented the production site within our simulation model, based on the automation pyramid framework outlined in section 2.1. The simulation model includes four core components: an ERP, an MES, a DT and cobots. Our primary focus is on the

communication between these systems, so we have deliberately excluded many of the functions typically associated with them. For example, the ERP is limited to sending production requests to the MES and hosting the DT, without handling tasks such as supply chain management, accounting, finance or logistics. These simplifications are intentional, as they allow us to focus on the communication between the modules rather than their detailed functionality. This approach is particularly important as our research focuses on identifying security threats within this communication. Similarly, we decided not to implement PLCs at the MS&O level, as their inclusion was not necessary to illustrate the interactions between the components of the simplified production site.

The production process starts in the ERP system, where the user enters a production order specifying the product and quantity to be produced. Once the order is placed, it becomes accessible to the MES system. In the current limited implementation of the ERP system, only two modules are realized: a simplified production planning module and a DT of the cobots. The goal of DT is to demonstrate how manipulated data can disrupt the foundation of modern businesses by corrupting essential operational data. Therefore, the focus of the simulation is not to make predictions, but to visualize and highlight deviations between the data-driven model and the actual workflows of the cobots. The data required for the simulation model is provided by the MES and contains only data about the cobots' activities, e.g. their positions, current tasks and status.

The MES serves as the production orchestration module, receiving production orders from the ERP system and translating them into specific production steps. In our use case, this translation focuses on the tasks assigned to the Cobots, particularly determining which materials they need to transport to the next production stations. Additionally, the MES ensures smooth communication with the ERP system by integrating with its reporting tools. It continuously monitors and reports on the progress and status of the cobots, relaying relevant information such as task completion and real-time operational updates back to the ERP system, ensuring the DT is aligned with the cobots.

The Cobots serve as the operational units that carry out the physical tasks, unlike the ERP and MES, which handle orchestration and administrative functions. On the virtual production site, the Cobots remain idle at their base stations until they receive a task from the MES. Once assigned, they begin transporting materials from the source to the next production stages, making multiple trips if necessary to move all required materials. After completing the task, they return to their base station. The Cobots regularly send status updates to the MES, including their position, state, and task progress. We assume that the Cobots always have sufficient battery charge to complete their tasks, as the focus of our simulation is on the communication channels and their security.

B. Technical Set-up

In this section, we present the technical aspects of the implementation of the use case specifically highlighting the communication protocols. Figure 2 shows the general relationship between the components and the communication protocols used. In the implementation three communication protocols are used: Hypertext Transfer Protocol Secure (HTTPs), Open Platform Communications Unified Architecture (OPC-UA) and direct communication of Python modules.

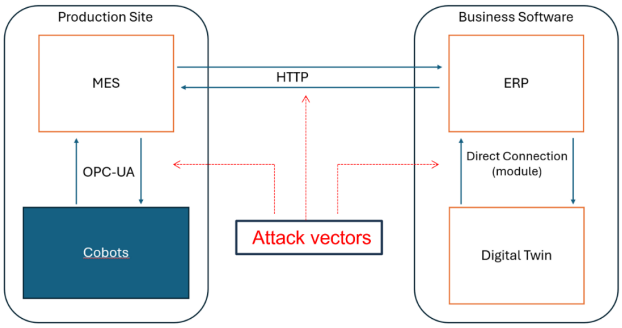

Fig. 2. The simulations communication architecture

The ERP system has two communication channels, a direct communication and a REST-API using HTTPs. The REST-API provides different endpoints consumed by the MES which are implemented in Python using the module flask. The endpoints implemented include reporting from the MES regarding the processes of the cobots and providing the MES with new production orders. In the first case the payload is sent to the ERP system from the MES and contains JSON data regarding the Cobots. In the second case the payload is sent to the MES from the ERP containing the production order descriptions as JSON data.

The MES establishes communication channels with the ERP and the individual cobots. To receive new production orders from the ERP system the MES polls the associated REST-API endpoint within predefined time intervals. Once a new production order has been received, it is translated into production commands the cobots can process, e.g. go to location *x* and transport *n* materials or containers to production steps *y*. Once the cobots update the MES regarding the tasks, the information is relayed to the ERP via the associated REST endpoint.

The cobots exclusively communicate with the MES system via OPC-UA. OPC-UA is a machine-to-machine communication protocol that ensures secure and reliable data exchange between industrial devices and systems. While the cobots are in an idle state, they are parked in their base stations and poll the OPC-UA server through a predefined method for new tasks. If a new task is received, the Cobots start to fulfill it. Once for every action taken by the Cobots, it reports their progress to the MES system via another OPC-UA method. These actions include moving, loading and unloading or start and finishing tasks.

IV. Attack Design

This section discusses our DoS attack design. First, it is worth noting that we considered both the communication between MES and ERP, which uses HTTP, and the communication between ERP and Digital Twin, which uses a direct connection, as insecure. This leads us to concentrate on the communication between MES and Cobots, which is deemed secure, instead of investigating all three communication protocols simultaneously. On the other hand, this can reveal some underlying behaviors and exploits of the OPC-UA when

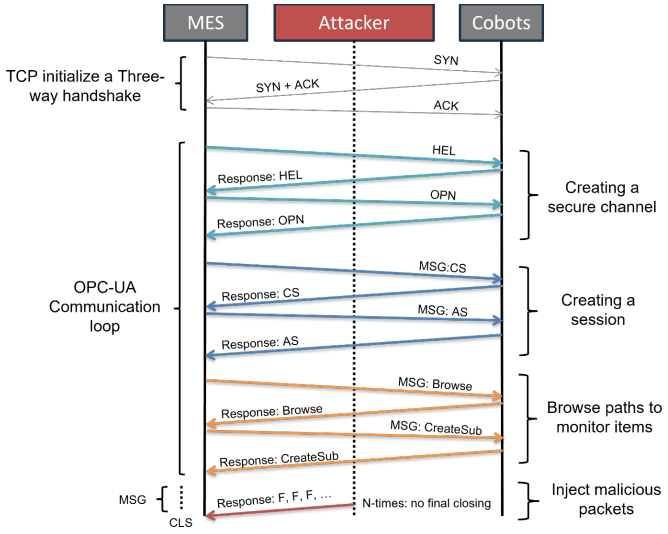

Fig. 3. OPC-UA communication session indicating our DoS exploit

communicating with other insecure protocols which can help with instigating vulnerabilities. Therefore, our main goal is focused on exploring the vulnerabilities in the communication between MES and Cobots based on OPC-UA, which is considered secure. As such, any potential network exploit that could lead to OPC-UA vulnerability could be detrimental to the future of the manufacturing industries, given their high dependence on open-source vendor implementations of the OPC-UA protocol [18].

Our attack design begins with the security analysis of the chosen OPC-UA protocol and how it interacts and exchanges information with other insecure protocols based on our simulation set-up in order to find vulnerabilities that can be exploited. This is mostly based on the analysis of fundamental functionalities of the OPC-UA stack [19], which in our case is the open-source python-opcua library [20]. Next, an exploit is detected and then expanded upon using tools such as Scapy to sniff all target addresses. Afterward, the target is further exploited by generating and injecting a malicious packet that can interfere with the communication in different scenarios. Thereby, inflicting irreparable damage.

A. Attack Strategy

This section analyzes the traffic in the communication protocol between MES and Cobots, which is based on OPC-UA.

1) *Exploiting unlimited final session*

The attack strategy of our approach leverages the TCP layer communication sequence of the OPC-UA to inject malicious packets using Scapy. Based on the assumption that an adversary has already successfully infiltrated the industrial network by bypassing user access, it becomes feasible to go straight to the target stage without needing to traverse each individual step in the process. First, our analysis begins by investigating traffic in a single communication session of the OPC-UA, as depicted in Fig. 3. The analysis reveals that the adopted python-opcua open-source library is open to multiple closing responses in CLO while the MES is awaiting response from the ERP via HTTP protocol before a transmission is terminated. However, it's important to note that the existing rogue client communication in our simulation setup, which is caused by an insecure protocol, exacerbates this issue. Specifically, after MSG transmission, the TCP layer takes over with the closing handshake from the closing communication established by the CLO. However, the handshake only ends when the TCP does not receive any closing message over a long period; otherwise, it keeps on waiting for the closing response. In essence, before the entire communication loop closes, the port remains open to TCP transactions, following the closure of the CLO, as captured by WireShark (see Fig. 4). Therefore, we leverage this delay to exploit the session even further by sniffing a suspicious, insecure MAC connected to the session and injecting malicious packets into it. The malicious packet is meant to overtake the next awaiting packet in the TCP closing loop of the session.

Consequently, this scenario in our simulation set-up means that until MES finishes sending the final payload to the COBOT, the CLO loop will always be open to receiving another packet until it is stopped. As such, the TCP layer carrying the final packet (right before the closing session) can easily be injected with a malicious payload in order to disrupt the communication.

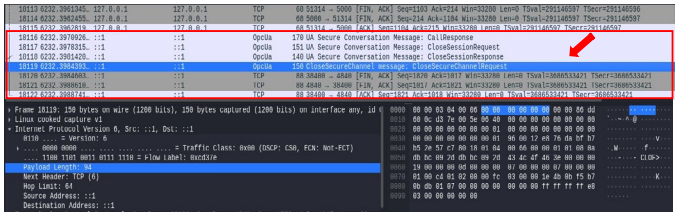

Fig. 4. OPC-UA closing session in Wireshark. Best viewed online. The arrow indicates the OpcUa closing session highlighted in blue. Bottom left in blue is the payload length, and bottom right is the data contained in the payload.

2) *Sniffing the network communication protocol*

This process involves sniffing suspicious IP and MAC addresses within the entire session using Scapy, especially those in the TCP layers of the final CLO. This helps reveal vital details regarding each sniffed IP and MAC address, enabling us to determine whether they are worth further exploration. During our investigation, we sniffed around 10 different MAC and IP addresses within the entire session and narrowed down to two suspicious addresses within the TCP closing session. And finally, one address is selected as our target MAC. In particular, the source and destination addresses, as well as the source and destination ports of the suspicious IP and MAC addresses, are analyzed during the exploration. Scapy is used for this analysis, while Wireshark is used to monitor any abnormal behaviors in the communication between the MES and COBOTs. Also, other system disruptions are monitored with the help of Wireshark. Fig. 5 depicts the packet sniffing of the target MAC and all information about the sniffed MAC.

3) *Generating and injecting malicious packet*

Once the packet has been sniffed, we proceed to generateand inject a new malicious packet to replace the previously sniffed one in the MAC. All this is done to ensure the target is executed during the closing session. Furthermore, the packet script is written in Python and designed to match what the server will expect from the client to acknowledge the closing session's final

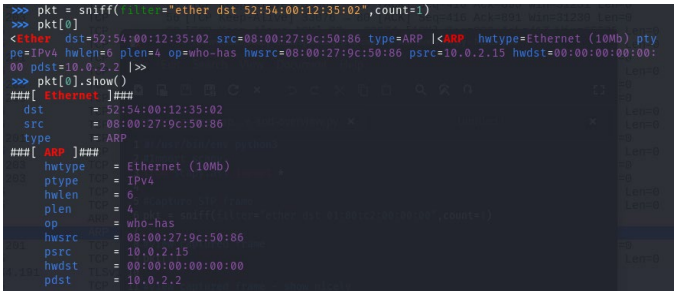

Fig. 5. Packet sniffing with Scapy. Best viewed online.

handshake. The malicious packet is injected using Scapy while monitoring the communication progress in Wireshark.

To thoroughly analyze every step of the packet injection process, our DoS starts with sending 50 packets at a time (see Fig. 6) and gradually increasing the number to 500 packets. Also, the looping begins at 0 with a sleep interval of one second. More details on this are given in the discussion section. Fig. 6 shows the looping snippet during the transmission of malicious packets.

V. Discussion and Mitigation

This section discusses the impact of our DoS attack on the entire communication protocol. The variant of the DoS attack in our analysis is especially interesting and dangerous because injecting a few packets only altered the data response in Cobots, which consequently led to altering the DT data but does not in any way impact the real system operations. This implies that detecting such an attack is challenging, as it does not interfere or disrupt the physical operations of the cobots but instead can lead to irreversible anomalies in the transmitted data. The data response alterations can be classified based on their impact in two major scenarios. 1) The data becomes unreliable, and 2) All data-driven applications, including DT, are impacted.

Our investigation also explored the possibility of simultaneously injecting a large number of malicious packets and increasing the packet loop. The outcome reveals the third impact: 3) System operations are disrupted for a relatively long time due to a delay in response. As such, the attack can be detrimental to both data and system reliability, depending on the attack target and the application context. Below is a brief description of our analysis based on the attack impact.

Fig. 6. Packet injection with Scapy

A. Data Unreliability and System Disruption

1) *Impact of data unrelaibility:* Our first analysis reveals the data response's unreliability as various changes occur to the actual transmitted payload. We used Wireshark to monitor and analyze these changes. In particular, after the malicious packet injection, the payloads in the three main packets are closely examined and recorded. These include a) the packet immediately before the malicious payload injection, b) the packets containing the injected malicious payload, and c) the packets immediately following the malicious payload injection.

a) *Packets before malicious payload injection*

Before the malicious payload is injected, the actual transmitted payload has a length of 32 with the header being db bc 09 2d. Also, it is worth noting that this payload resides within the IP protocol of the TCP/IP. Therefore, even though it appears from our side that it has nothing to do with the sniffed MAC address, the impact remains given that it falls within the final responses to the closing session where the malicious payload is injected. Figure 7(a) shows the communicated packet and its payload before the malicious packet injection.

b) *Packets with malicious payload*

The infected packet in Figure 7(b) shows an erroneous/false TCP retransmission, indicating a misbehavior in the packet transmission. First, it can be noticed that the payload of the packet appears in the TCP layer of the OPC-UA protocol, indicating a change in transmission layer after the infection with a TCP payload of 39 bytes. Additionally, the message header appears significantly larger than the payload prior to the injection of a malicious packet. This means that an additional data payload from the malicious packet is added in the transmission.

c) *Packets after malicious payload injection*

After the malicious payload injection, packet transmission is expected to come back to normal as it was prior to the malicious payload injection, with the header information *db bc 09 2d*. In this case, however, we can see in Figure 7(c) that the packet transmits a payload that is entirely different from the initial one, with only an information header of *02*. Also, the payload length has increased from 32 to 40, which indicates that the data payload from the injected malicious packet is added to each subsequent packet even after transmission but without any noticeable disruption in the system operations. This indicates the significant impact of the attack, along with the inconsistency and unreliability in all subsequent data transmissions across the entire communication protocol.

Based on this analysis, we can conclude that the injected packet has altered the original transmission, resulting in highly unreliable subsequent data transfers between all applications in our setup. This leads us to the second impact of the attack on data-driven applications.

2) *Imapct on data driven applications:* Given that all the applications within our communication simulation set-up, particularly the DT, rely on data, it means that these applications will consequently be impacted through the retransmission of rogue data. In this scenario, the DT's inability to mirror and replicate the physical activities of the Cobots will result in erroneous data being fed to other systems, thereby interfering with the entire operation due to malicious data transmission. This scenario is known as knock-on effect.

3) *System response delay and total disruption:* To further investigate the attack impact at the system level, we considered

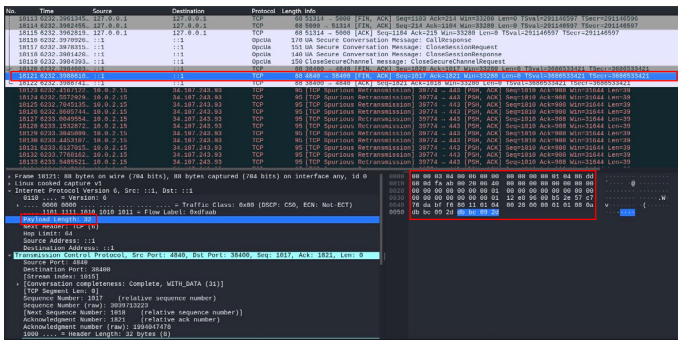

Fig. 7(a). TCP packet before malicious payload injection. Best viewed online. The top highlighted in blue is the TCP packet before injecting malicious payload. Bottom left in blue is the payload length, and bottom right is the data contained in the payload.

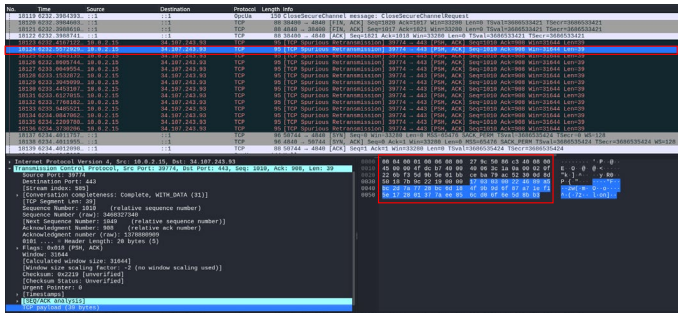

Fig. 7(b). TCP packet with malicious payload injection. Best viewed online. Top highlighted in blue is the TCP packet with injected malicious payload. Bottom left in blue is the payload data, and bottom right in blue is the additional malicious data contained in the payload. Notice an increase in the TCP payload to 39 bytes.

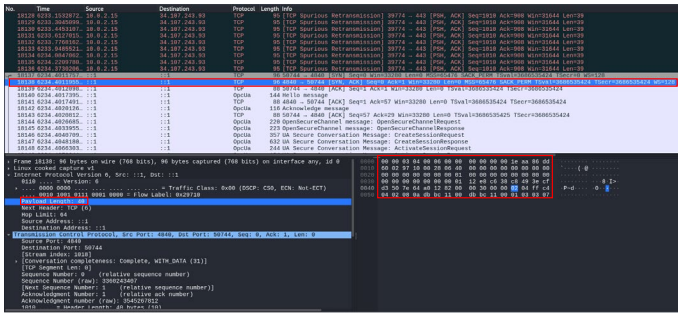

Fig. 7(c). TCP packet after malicious payload injection. Best viewed online. The top highlighted in blue is the TCP packet after the payload injection. Bottom left in blue is the payload length, and bottom right is the data contained in the payload. Notice the increase in the payload length from 32 to 40 when compared to the payload length before malicious data injection.

increasing the number of malicious packets from 50 to 500. We observed that injecting between 50 to 150 packets within a 1- or 2-second interval only leads to data unreliability, as discussed in points 1) and 2) above. However, increasing the packet injection loop while reducing the sleep time not only impacts data unreliability, but also causes a delay in response, leading to a total disruption of system operations over time. It can be seen in Table 1 that a higher packet looping of 50 or 100 can lead to disruptions in the system operations due to the high response delay. Consequently, this prolonged delay will cause the system to crash. Furthermore, we observe that the delay

Table 1. Packet injection loop test

No.	No. of sent packets	Loop time	Packet looping	Response delay	Attack impact
1	50	1 sec	0 loop	51 seconds	Medium/High
2	100	1 sec	2 loops	4.98 minutes	Medium/High
3	150	1.5 sec	50 loops	10.5 minutes	Medium/High
4	200	500 msec	100 loops	22.4 minutes	High
5	300	300 msec	100 loops	28.8 minutes	High
6	400	100 msec	50 loops	32.6 minutes	High
7	500	50 msec	50 loops	35.7 minutes	High

response escalates as the number of packets increases, despite no increase in the looping time or packet looping frequency, which indicates that increasing the packet loop will only further increase the response delay.

Moreover, depending on the attack aim, all impacts can be considered high because even the injection of 50 malicious packets gets the data unreliable, which can be more dangerous than system response delay or disruption depending on the application context of the attack. Figure 8 depicts a total system disruption captured in Wireshark and Scapy due to the injection of 100 packets at 100 loops.

B. Mitigation Strategies

To mitigate against this variant of DoS attack. We propose four key mitigation strategies as given below.

1) Vendors should adhere to OPC-UA foundation's specification: The foundation's specification remains a crucial arsenal that must be followed and adhered to during the OPC-UA implementation. The OPC-UA manual provides a comprehensive explanation of the various specifications and configuration requirements that must be complied with during the OPC-UA implementation. It is recommended that developers thoroughly read those specifications before implementing the OPC-UA protocol because any deviation from the specified configurations can lead to security issues.

2) Avoid a set-up that combines OPC-UA with other insecure protocols: It is vital to ensure that the set-up of an entire network does not include insecure protocols that can be compromised. This is because any of the protocols can execute commands and transmit data at any given time within the entire communication network, thereby revealing exploits that could lead to compromising other entities. As indicated in our simulation set-up, having a vulnerable HTTP or direct connection within the network can lead to compromising the entire communication by introducing a rogue server or client to the OPC-UA protocol.

3) Vendors should only implement OPC-UA recommended by the OPC-UA foundation: Given the different publicly available OPC-AU artifacts across different products and libraries, security issues have been identified in most of the artifacts [3]. Therefore, it is imperative that vendors fully rely on OPC-UA products and libraries recommended by the foundation, as these products and libraries incorporate all security features according to the foundation's specifications.

4) Integrate additional security defense mechanisms: In situations where it is inevitable to have other insecure protocols

in the communication network, it is crucial to adopt additional layers of defense, such as authentication and encryption, as well as intrusion detection systems to ensure that the communication remains secure [21].

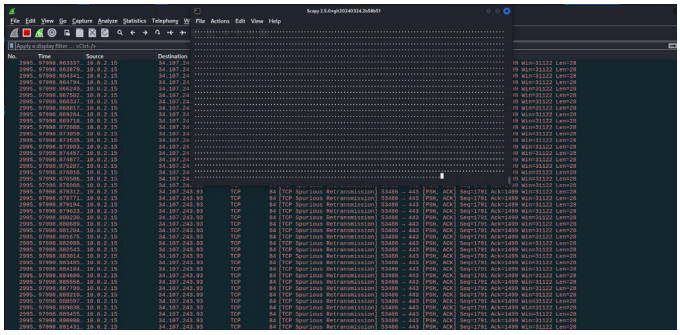

Fig. 8. Packet injection at 100 looping causing total system collapse. Best viewed online. Notice that all TCP packets turns red, indicating spurious retransmission. The middle pane indicates looping by Scapy.

VI. Conclusion and Future Work

In this work, we analyzed the OPC-UA communication protocol between MES and Cobots using our simulated communication architecture. The communication architecture mimics most vendor implementations with both secure and insecure protocols combined. It also constitutes the most dominant CPS elements including, DT, Cobots, MES, and ERP. Based on the analysis of the communication protocol, a suspicious MAC address is detected, sniffed, and injected with a malicious packet. Thus, disrupting the system operation through the transmission of unreliable data between the different entities in the network. Moreover, as the number of packet injections increases with an increase in the number of loops, the system crashes, and all operations are halted. To avert this vulnerability, we proposed different mitigation strategies for vendor implementations of OPC-UA.

Our novel findings reveal significant security vulnerabilities when OPC-UA is configured with other insecure protocols, thereby jeopardizing its security guarantees. The findings also demonstrate that many OPC UA deployments in the industry fail to provide complete security guarantees when authenticating processes between different network protocols. As such, we believe that our proposed analysis is beneficial in raising awareness among companies, users, and developers regarding the potential threats in the OPC-UA vendor implementations, thereby emphasizing the necessity for strong security measures to protect critical infrastructure from cyber threats.

Aknowledgement

This work is part of the funding received from the ONE4ALL project, funded by the European Commission, Horizon Europe Program under the Grant Agreement No. 101091877.

References

[1] A. E. Elhabashy, L. J. Wells, and J. A. Camelio, "Cyber-Physical Security Research Efforts in Manufacturing – A Literature Review," *Procedia Manufacturing,* vol. 34, pp. 921-931, 2019, doi: https://doi.org/10.1016/j.promfg.2019.06.115.

[2] C. Parian, T. Guldimann, and S. Bhatia, "Fooling the Master: Exploiting Weaknesses in the Modbus Protocol," *Procedia Computer Science,* vol. 171, pp. 2453-2458, 2020, doi: https://doi.org/10.1016/j.procs.2020.04.265.

[3] A. Erba, A. Müller, and N. O. Tippenhauer, "Security Analysis of Vendor Implementations of the OPC UA Protocol for Industrial Control Systems," presented at the Proceedings of the 4th Workshop on CPS & IoT Security and Privacy, Los Angeles, CA, USA, 2022. [Online]. Available: https://doi.org/10.1145/3560826.3563380.

[4] A. Ocaka, D. Ó. Briain, S. Davy, and K. Barrett, "Cybersecurity Threats, Vulnerabilities, Mitigation Measures in Industrial Control and Automation Systems: A Technical Review," in *2022 Cyber Research Conference - Ireland (Cyber-RCI)*, 25-25 April 2022, pp. 1-8, doi: 10.1109/Cyber-RCI55324.2022.10032665.

[5] S. M. Abdullahi and S. Lazarova-Molnar, "Cybersecurity in Distributed Industrial Digital Twins: Threats, Defenses, and Key Takeaways," presented at the 1st International Workshop on Distributed Digital Twins, Groningen, The Netherlands, 2024.

[6] A. Martins, J. Lucas, H. Costelha, and C. Neves, "CNC Machines Integration in Smart Factories using OPC UA," *Journal of Industrial Information Integration,* vol. 34, p. 100482, 2023, doi: https://doi.org/10.1016/j.jii.2023.100482.

[7] C. Team82. "OPC UA Deep Dive: A Complete Guide to the OPC UA Attack Surface." https://claroty.com/team82/research/opc-ua-deep-dive-a-complete-guide-to-the-opc-ua-attack-surface (accessed August, 2024).

[8] C. Team82. "OPC UA Deep Dive Series (Part 6): A One-of-a-Kind OPC UA Exploit Framework." https://claroty.com/team82/research/opc-ua-deep-dive-series-a-one-of-a-kind-opc-ua-exploit-framework (accessed August, 5, 2024).

[9] C. V. Neu, I. Schiering, and A. Zorzo, "Simulating and Detecting Attacks of Untrusted Clients in OPC UA Networks," presented at the Proceedings of the Third Central European Cybersecurity Conference, Munich, Germany, 2019. [Online]. Available: https://doi.org/10.1145/3360664.3360675.

[10] N. Mühlbauer, E. Kirdan, M. O. Pahl, and G. Carle, "Open-Source OPC UA Security and Scalability," in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 8-11 Sept. 2020 2020, vol. 1, pp. 262-269, doi: 10.1109/ETFA46521.2020.9212091.

[11] C. Team82. "OPC UA Deep Dive (Part 1 & 2): History of the OPC UA Protocol." https://claroty.com/team82/research/opc-ua-deep-dive-history-of-the-opc-ua-protocol (accessed July, 2024).

[12] F. O. UA. "OPC-UA implementation." https://github.com/FreeOpcUa/opcua-asyncio (accessed August, 15.

[13] "IEC 62264 - Enterprise-control system integration." https://www.iso.org/standard/57308.html (accessed 10.09.024.

[14] M.-F. Körner *et al.*, "Extending the Automation Pyramid for Industrial Demand Response," *Procedia CIRP,* vol. 81, pp. 998-1003, 2019, doi: 10.1016/j.procir.2019.03.241.

[15] S. M. Abdullahi and S. Lazarova-Molnar, "On the adoption and deployment of secure and privacy-preserving IIoT in smart manufacturing: a comprehensive guide with recent advances," *International Journal of Information Security,* vol. 24, no. 1, p. 53, 2025, doi: 10.1007/s10207-024-00951-8.

[16] S. Brodie and T. Oksanen, "OPC UA client-server connection over an ISO 11783 vehicle network," vol. 71, no. 11, pp. 916-927, 2023, doi: doi:10.1515/auto-2023-0030.

[17] O. Foundation. "OPC Unified Architecture Specification." https://opcfoundation.org/about/opc-technologies/opc-ua/ (accessed.

[18] D.-H. Shin, G.-Y. Kim, and I.-C. Euom, "Vulnerabilities of the Open Platform Communication Unified Architecture Protocol in Industrial Internet of Things Operation," *Sensors*, vol. 22, no. 17, doi: 10.3390/s22176575.

[19] J. T. D. Silva, A. L. Dias, and I. N. D. Silva, "A Survey on OPC UA Protocol: Overview, Challenges and Opportunities," in *2023 15th IEEE International Conference on Industry Applications (INDUSCON)*, 22-24 Nov. 2023, pp. 1523-1530, doi: 10.1109/INDUSCON58041.2023.10375053.

[20] A. H. and Oroulet. "FreeOpcUa." https://github.com/FreeOpcUa/opcua-asyncio (accessed July 5, 2024).

[21] S. M. Abdullahi and S. Lazarova-Molnar, "Toward a Unified Security Framework for Digital Twin Architectures," in *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, 2-4 Sept. 2024 2024, pp. 612-617, doi: 10.1109/CSR61664.2024.10679442.